Rockchip RT-Thread I2C Developer Guide

ID: RK-KF-YF-113

Release Version: V1.1.0

Release Date: 2024-12-26

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

Product Version

Chipset	RT-Thread Version
ALL	ALL

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Wu Dachao	2020-02-18	Initial Version
V1.0.1	Chen Mou Chun	2020-03-17	Correction of Links
V1.0.2	Chen Mou Chun	2020-05-27	Correction of Format
V1.1.0	Wu Dachao	2024-12-26	Added content to adapt to all chips

Contents

Rockchip RT-Thread I2C Developer Guide

1. Feature Configuration
2. Board-Level Configuration
3. Code and API Usage
   - 3.1 Code Location
   - 3.2 I2C API Interface
     - 3.2.1 Finding I2C Bus Devices
     - 3.2.2 I2C Write
     - 3.2.3 I2C Read
   - 3.3 Developer Guide
   - 3.4 I2C CMD Testing
4. Debugging
   - 4.1 Peripheral Unresponsive
   - 4.2 Timeout

1. Feature Configuration

The functional configuration of I2C is divided into two parts: the driver framework and VENDOR configuration. The specific configurations required are as follows:

```
# I2C Driver Framework
RT_USING_I2C [=y]
# VENDOR Driver, enable the I2C bus you need according to your requirements
RT_USING_I2C0 [=y]
RT_USING_I2C1 [=y]
RT_USING_I2C2 [=y]
```

2. Board-Level Configuration

- board.c

In the board-level configuration file, include the header file and the I2C bus configuration table to configure the bus speed, which currently supports 100K, 400K, and 1000K (when configuring the speed, please ensure that the rise time meets the timing specifications for that speed mode), other speeds need to be added manually. An example of board-level configuration is as follows:

```
#ifdef RT_USING_I2C
#include "drv_i2c.h"
#endif

#ifdef RT_USING_I2C
const struct rockchip_i2c_config rockchip_i2c_config_table[] =
{
#ifdef RT_USING_I2C0
    {
        .id = I2C0,
        .speed = I2C_400K,
    },
#endif
    { /* sentinel */ }
};
#endif
```

- iomux.c

The I2C IOs are all multiplexed, so it is necessary to ensure that the relevant IOs in the IOMUX configuration are switched to I2C mode. For details, please refer to the IOMUX section in the Developer Guide.

3. Code and API Usage

3.1 Code Location

- Controller Driver Layer Code: ./bsp/rockchip/common/driver/i2c/drv_i2c.c
- Controller HAL Layer Code: ./bsp/rockchip/common/hal/lib/hal/src/hal_i2c.c

3.2 I2C API Interface

3.2.1 Finding I2C Bus Devices

```
#define I2C_BUS_NAME      "i2c1"        /* I2C Bus Device Name */
struct rt_i2c_bus_device *i2c_bus;      /* I2C Bus Device Handle */

/* Find the I2C bus device and obtain the I2C bus device handle */
i2c_bus = (struct rt_i2c_bus_device *)rt_device_find(name);
if (i2c_bus == RT_NULL)
  return -RT_ERROR;
```

3.2.2 I2C Write

```
static rt_err_t i2c_write(struct rt_i2c_bus_device *bus, rt_uint8_t addr, rt_uint8_t reg,
rt_uint8_t *data)
{
    rt_uint32_t buf_size = 1;
    struct rt_i2c_msg msgs;
    rt_uint8_t buf[3];

    buf[0] = reg; //cmd
    if (data != RT_NULL)
    {
        buf[1] = data[0];
        buf[2] = data[1];
        buf_size = 3;
    }

    msgs.addr = addr;
    msgs.flags = RT_I2C_WR;
    msgs.buf = buf;
    msgs.len = buf_size;

    /* Call the I2C device interface to transfer data */
    if (rt_i2c_transfer(bus, &msgs, 1) == 1)
    {
        return RT_EOK;
```

```
    }
    else
    {
        return -RT_ERROR;
    }
}
```

3.2.3 I2C Read

```
static rt_err_t i2c_read(struct rt_i2c_bus_device *bus,  rt_uint8_t addr,  rt_uint8_t len,
rt_uint8_t *buf)
{
    struct rt_i2c_msg msgs;

    msgs.addr = addr;
    msgs.flags = RT_I2C_RD;
    msgs.buf = buf;
    msgs.len = len;

    /* Call the I2C device interface to transfer data */
    if (rt_i2c_transfer(bus, &msgs, 1) == 1)
    {
        return RT_EOK;
    }
    else
    {
        return -RT_ERROR;
    }
}
```

3.3 Developer Guide

The I2C driver adheres to the RTT system standard I2C driver framework, so you can directly refer to the RTT official I2C Developer Guide.

3.4 I2C CMD Testing

I2C CMD code is located in components/drivers/i2c/i2c_cmd.c

It has the same usage as Linux i2c tools, supporting CMDs including: i2cdetect, i2cdump, i2cget, i2cset. For example, the usage of i2cdetect is as follows:

```
# i2cdetect -y 0
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- 51 -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

4. Debugging

4.1 Peripheral Unresponsive

Return value HAL_NODEV = (-19), indicating the peripheral is unresponsive NACK,

- It is likely that the device address is incorrect, as the address does not include read/write bits.
- There may be issues with the power supply voltage or power-up sequence for the I2C peripheral device, preventing it from functioning properly.

4.2 Timeout

Common causes of timeout issues include:

- The I2C pull-up power supply is missing, or the pull-up resistor voltage is incorrect, not matching the IO voltage, such as insufficient voltage, etc.
- The iomux configuration is not normal.
- The application reuses the pin repeatedly, causing abnormal operation.
- The peripheral pulls down the I2C bus.